



Planning d'équipe intelligent — solver d'auto-planning & sync calendrier natif

Dossier de projet réalisé dans le cadre de la présentation au
Titre Professionnel Développeur Web & Web Mobile

RNCP 37674 — Niveau 5

Vladimir Rodzaevskiy

Projet individuel · Démo : crew-planner-hazel.vercel.app

Sommaire

Introduction	3
Liste des compétences couvertes par le projet	4
I. Front-end — application web sécurisée	4
II. Back-end — application web sécurisée	5
Résumé du projet	6
Cahier des charges	7
Besoins, objectifs, concepts métier	7
User stories · Arborescence · MVP	9
Rôles & permissions · Maquettes de l'application	11
Spécifications techniques	13
Technologies	13
Base de données — MCD / MLD / dictionnaire	14
API REST — routes	17
Extraits de code commentés	19
Sécurité — analyse OWASP	22
Plan de tests & jeu d'essai	24
Veille technique & exemple de raisonnement	26
Évolutions — solver enrichi (migrations 006-012)	28
Conclusion	31

Introduction

J'ai choisi de réaliser **Crew**, une application web responsive de planification de services d'équipe. L'idée vient d'un constat simple et concret : dans la restauration, l'hôtellerie ou le retail, le manager passe un temps considérable à bâtir chaque semaine un planning équitable, qui respecte les heures contractuelles de chacun, la couverture nécessaire de chaque service et le droit du travail. Ce travail est répétitif, source d'erreurs et difficile à diffuser proprement aux équipiers.

Crew répond à ce besoin avec un **solver d'auto-planning** : le manager configure une fois le profil de chaque équipier (poste, créneau habituel, heures cibles), puis génère en un clic un planning hebdomadaire équilibré qu'il peut ajuster avant publication. Chaque équipier reçoit ensuite ses services directement dans le calendrier de son téléphone via un flux iCal personnel, sans aucune application à installer.

J'ai mené ce projet de façon individuelle et autonome, du cahier des charges aux maquettes, puis au développement front-end et back-end, jusqu'au déploiement en production. Il m'a confronté à des défis techniques réels — modélisation de la base, conception d'un algorithme d'affectation sous contraintes, sécurisation de l'API — qui sont au cœur des compétences du titre et que ce dossier détaille.

Liste des compétences couvertes par le projet

Le titre RNCP 37674 est structuré en deux activités-types. La sécurité n'est pas un bloc séparé : elle est exigée à l'intérieur de chacune (« application web ou web mobile **sécurisée** »). Chaque compétence ci-dessous renvoie à la section du dossier qui en apporte la preuve.

I. Développer la partie front-end d'une application web sécurisée

A. Maquetter une application. En amont du développement, j'ai formalisé les besoins via des *user stories* (utilisateur / manager / admin), une arborescence de navigation, un MVP et des maquettes des écrans clés. → *Cahier des charges, p. 9-12.*

B. Réaliser une interface utilisateur web statique et adaptable. L'interface est développée en React 18 + Vite, découpée en composants réutilisables, stylée en CSS pur (custom properties) pour un rendu responsive desktop/mobile, et internationalisée (FR/EN) via i18next. Les bonnes pratiques d'accessibilité sont appliquées (HTML sémantique, contrastes, navigation clavier). → *Spécifications techniques p. 13 ; Sécurité front p. 22.*

C. Développer une interface utilisateur web dynamique. Gestion d'état via Context API (auth, équipe, thème, toasts), routage protégé (ProtectedRoute), appels API authentifiés par JWT, modaux interactifs (preview du smart planner), graphiques Chart.js. Les formulaires valident les saisies et n'exposent aucun secret côté client. → *Extraits de code p. 19 ; Sécurité p. 22.*

II. Développer la partie back-end d'une application web sécurisée

A. Créer une base de données. Conception du MCD puis du MLD, écriture des migrations SQL versionnées (MariaDB 10.11), définition des contraintes d'intégrité (clés étrangères, unicité, cascades) et d'un dictionnaire de données complet. → *Spécifications techniques p. 14-16.*

B. Développer les composants d'accès aux données. Couche Model en SQL paramétré (driver mysql2, requêtes préparées contre l'injection), organisée en modèles (user, team, teamMember, shift), pool de connexions, migrations et seed. → *Extraits de code p. 20 ; Sécurité p. 23.*

C. Développer la partie back-end (métier & sécurité). API REST Express structurée en MVC (routes → controllers → services → models), validation systématique côté serveur, authentification JWT + bcrypt, contrôle d'accès par rôle (RBAC) via middleware sur chaque endpoint, rate-limiting, CORS restrictif, et la logique métier centrale : le **solver d'auto-planning** sous contraintes légales (Convention HCR / Code du travail). → *Extraits de code p. 19-21 ; Sécurité p. 22-23 ; Évolutions p. 28.*

Résumé du projet

Crew est une application web responsive de planification de services d'équipe, dédiée à tout responsable gérant une rotation de shifts : restauration, hôtellerie, retail, santé, sécurité, événementiel.

Objectifs

- **Objectif général** — réduire le temps et les erreurs de la planification hebdomadaire en la rendant assistée, équitable et conforme au droit du travail.
- Proposer un **planning automatique** en un clic à partir du profil des équipiers.
- Offrir une **expérience fluide** au manager (preview, ajustement, publication, statistiques).
- Diffuser les services aux équipiers via un **calendrier natif**, sans application à installer.
- Garantir les **bonnes pratiques de sécurité** à chaque niveau de l'application.

Fonctionnalités principales

- **Compte & équipes** — inscription/connexion, création d'équipe, adhésion par code d'invitation, modération des nouvelles demandes, rôles manager/équipier/admin.
- **Smart Planner** — génération automatique d'un planning hebdomadaire, preview chiffrée (heures par membre, slots non couverts, masse salariale), clone-week / clear-week.
- **Planning** — grille membres × jours, création/édition de shifts, indicateurs de couverture.
- **Statistiques** — répartition par poste, par membre, timeline (Chart.js).
- **Calendrier natif** — flux iCal personnel et par équipe, alarme VALARM 2 h avant chaque service, QR code d'abonnement.

Architecture & stack

- **Front** : React 18 + Vite + React Router 6, i18next (FR/EN), Chart.js, CSS pur.
- **Back** : Node.js 22 + Express 4, mysql2, architecture MVC.
- **Base de données** : MariaDB 10.11.
- **Sécurité** : JWT HS256, bcrypt, RBAC par middleware, CORS strict, rate-limiting, requêtes préparées.
- **Déploiement** : Vercel (front) + Fly.io (back + base dans un conteneur Docker).

Limites actuelles & suites prévues

- Densité unique par service (pas de granularité intra-service) — assumé, voir Évolutions.
- Tests automatisés à étendre (couverture manuelle documentée pour l'instant).
- Pistes : notifications push, suggestion automatique de densité depuis l'historique, calibration de productivité par paire (membre, poste).

Cahier des charges

I. Besoins et objectifs

Constats

- La planification manuelle (tableur, papier) est chronophage et source d'erreurs : sur-staffing coûteux, sous-couverture risquée, dépassements du droit du travail.
- La diffusion du planning aux équipiers est peu fiable (photo d'un tableau, message groupé).
- Le suivi des heures contractuelles et de la masse salariale est rarement outillé en temps réel.

Solution

- Un **solver** qui propose un planning équitable respectant postes, créneaux, heures cibles et contraintes légales.
- Un **flux calendrier natif** personnel par équipier.
- Des **indicateurs** de couverture, d'heures et de coût directement dans l'interface.

II. Concepts métier

Utilisateur

Compte personnel (email + mot de passe). Peut créer une équipe ou en rejoindre une via un code d'invitation, et appartenir à plusieurs équipes (ex : un serveur travaillant dans deux restaurants).

Équipe

Groupe d'utilisateurs partageant un planning. Chaque membre a un rôle (`manager` ou `équipier`). Un manager administrateur peut inviter, valider et retirer des membres.

Membre d'équipe — caractéristiques métier

- **Poste** — zone fonctionnelle (cuisine, salle, bar, plonge, administration). Permet au solver d'affecter les bons profils aux bons services.
- **Shift habituel** — créneau préféré (matin, midi, coupure, soir, nuit). Indice de score.
- **Heures hebdomadaires cibles** — quota contractuel (35 h, 24 h...). `NULL` ou `0` = membre exclu du planning automatique (cadres, patron).

Shift

Créneau de travail planifié pour un équipier donné. Contrainte d'unicité (`user_id, date, shift_type`) : pas de double-planning sur le même créneau d'un jour.

Smart Planner

Algorithme greedy à score multicritères :

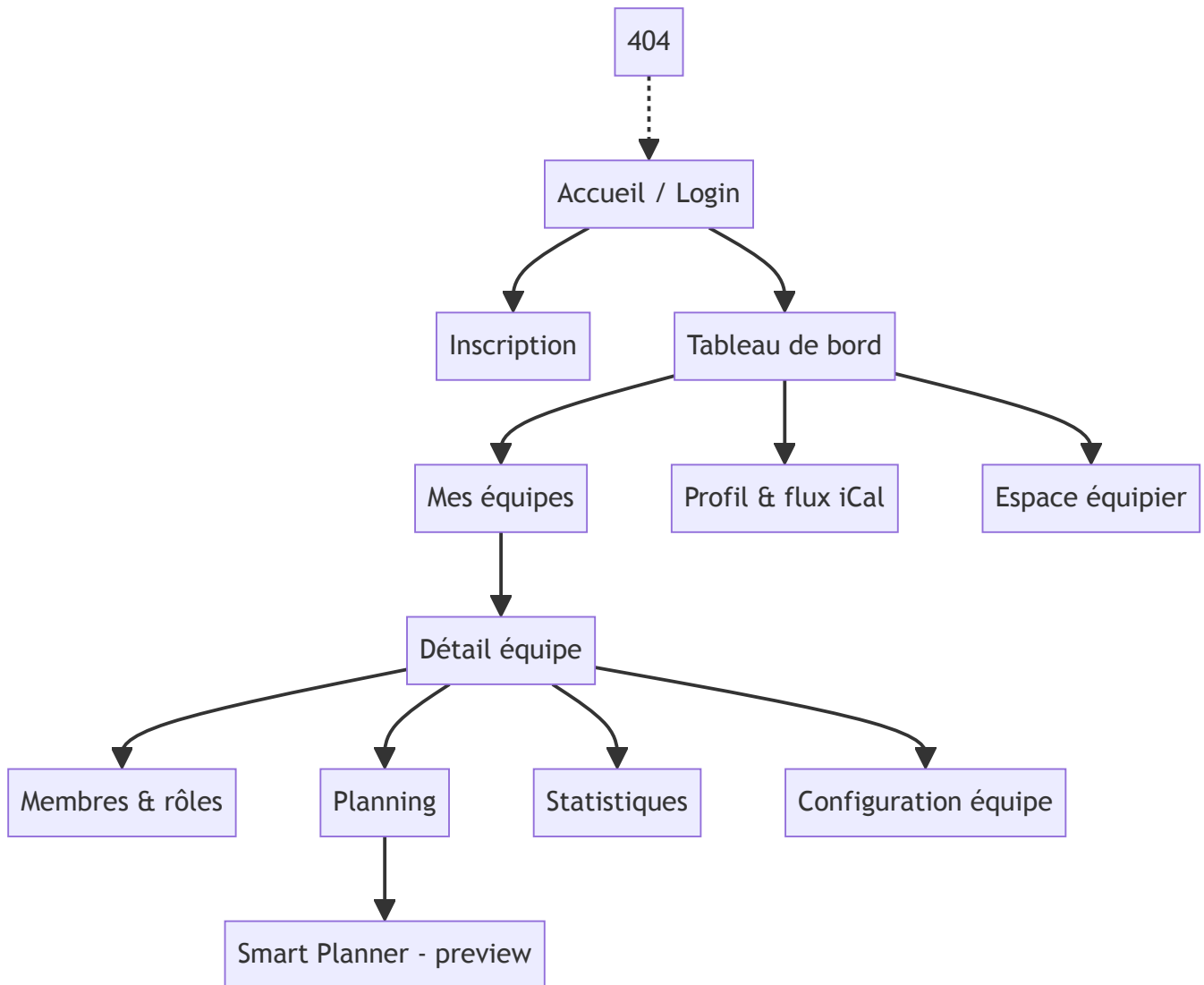
1. Préférer les membres dont les heures planifiées sont les plus éloignées (en dessous) de leur cible.
2. Bonus si le poste demandé correspond au poste habituel.
3. Bonus si le créneau demandé correspond au shift habituel.
4. Malus à partir du 5^e jour consécutif ; rejet strict au-delà de 6 jours (Code du travail).

Le manager voit la proposition dans un modal de preview (heures par membre, slots non couverts, total) avant de l'appliquer.

III. User stories

En tant que	Je souhaite	Afin de
Utilisateur	m'inscrire et me connecter	accéder à l'application en sécurité
Utilisateur	créer une équipe ou en rejoindre une par code	partager un planning
Manager	configurer le profil d'un équipier (poste, créneau, heures)	alimenter le solver
Manager	générer un planning automatique	gagner du temps et de l'équité
Manager	prévisualiser puis ajuster la proposition	garder le contrôle final
Manager	cloner ou effacer une semaine	accélérer la mise en place
Manager admin	inviter, valider, retirer des membres	administrer l'équipe
Manager admin	réinitialiser le mot de passe d'un membre	dépanner un équipier
Équipier	consulter mes prochains shifts	m'organiser
Équipier	m'abonner au flux iCal	recevoir mes services dans mon téléphone
Manager	consulter les statistiques d'équipe	piloter l'activité

IV. Arborescence de navigation



Accès commun (tous rôles) : profil, abonnement iCal, consultation du planning. Accès manager : édition des shifts, smart planner. Accès admin : gestion des membres et de l'équipe.

V. MVP

Le « Produit Minimum Viable » vise une planification fiable et conforme :

- Authentification sécurisée, création/adhésion d'équipe par code.
- Configuration des équipiers et génération d'un planning auto (preview + application).
- Grille de planning éditable et flux iCal personnel.

Hors MVP (évolutions) : statistiques avancées, multi-skill, cost-aware, alertes science-based (livrées ensuite — voir section Évolutions).

VI. Rôles et permissions

Action	Manager admin	Manager	Équipier
Inviter / approuver un membre	✓	✗	✗
Modifier rôle / poste / heures	✓	✗	✗
Retirer un membre	✓	✗	✗
Régénérer le code d'invitation	✓	✗	✗
Réinitialiser le mot de passe d'un membre	✓	✗	✗
Créer / éditer / supprimer un shift	✓	✓	✗
Générer / appliquer un smart planning	✓	✓	✗
Voir tout le planning de l'équipe	✓	✓	✓
Consulter les statistiques équipe	✓	✓	✗
Voir ses propres shifts	✓	✓	✓
S'abonner au flux iCal	✓	✓	✓

VII. Maquettes (captures de l'application)

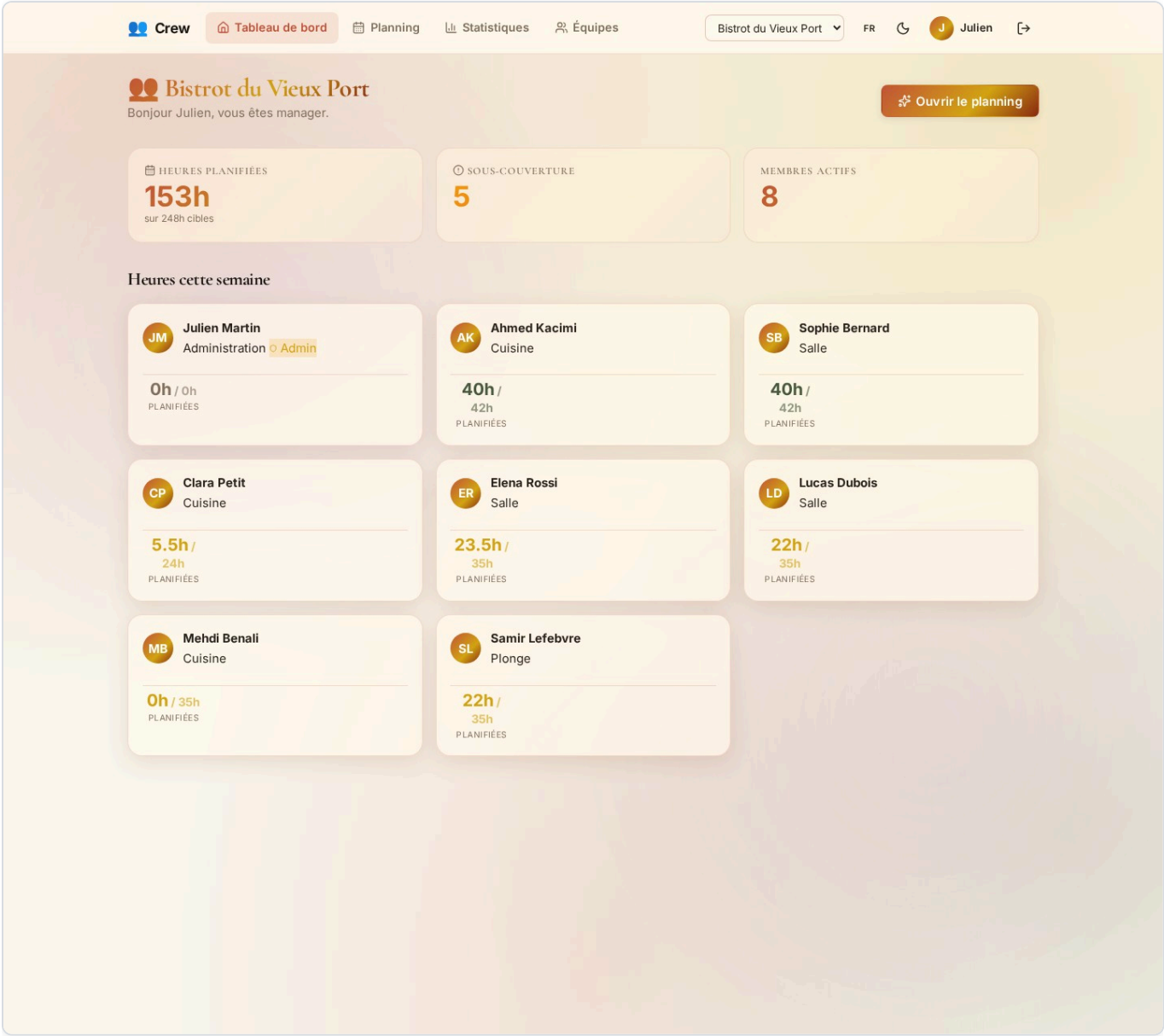


Tableau de bord du manager — heures planifiées par équipier vs cibles, couverture par poste, alertes santé du service.

Crew	Tableau de bord	Planning	Statistiques	Équipes	Bistrot du Vieux Port	FR	Julien	
Planning de service					Cette semaine	Générer		
⚠ 15 créneau(x) nécessitent un extra cette semaine dim. 31 Midi Cuisine +100 dim. 31 Midi Salle +100 dim. 31 Soir Cuisine +100 dim. 31 Soir Salle +100 mar. 2 Midi Cuisine +100 mar. 2 Midi Salle +100 mar. 2 Soir Cuisine +100 mar. 2 Soir Salle +100 + 7								
Du 1 juin au 7 juin 2026 · Masse salariale : 1934.00 €								
Équipier	Lun 1 ●● M 0.00 / S 0.00	Mar 2	Mer 3 ●● M 0.00 / S 0.00	Jeu 4 ●● M 0.00 / S 0.00	Ven 5 ●● M 0.80 / S 1.00	Sam 6 ●● M 0.80 / S 1.28	Dim 7 ●● M 0.80 / S 1.00	
Q CUISINE								
Ahmed Cuisine 30h / 42h	+	+	+	+	Midi Cuisine Soir Cuisine	Midi Cuisine Soir Cuisine	Midi Cuisine Soir Cuisine	
Clara Cuisine 5.5h / 24h	+	+	+	+	+	Soir Cuisine +	+	
Mehdi Cuisine 0h / 35h	+	+	+	+	+	+	+	
Samir Plonge 16.5h / 35h	+	+	+	+	Soir Plonge +	Soir Plonge +	Soir Plonge +	
Extras manque sur ce poste	+100 Midi +100 Soir		+100 Midi +100 Soir	+100 Midi +100 Soir	+40 Midi	+40 Midi	+40 Midi	
Q SALLE								
Elena Salle 19h / 35h	+	+	+	+	Midi Salle +	Midi Salle Soir Salle	Midi Salle +	
Lucas Salle 16.5h / 35h	+	+	+	+	Soir Salle +	Soir Salle +	Soir Salle +	
Sophie Salle 30h / 42h	+	+	+	+	Midi Salle Soir	Midi Salle Soir	Midi Salle Soir	

Grille de planning hebdomadaire — équipiers groupés par poste (cuisine, salle), ligne « Extras » avec déficits à couvrir, indicateurs de couverture par service en en-tête, masse salariale prévisionnelle affichée.

Planificateur automatique

Pour la semaine du 31 mai au 6 juin

Densité prévue de chaque service. 1,0 = standard, 0,5 = calme, 1,3 = chargé.

Jour

Tous

dim. 31

lun. 1

mar. 2

mer. 3

jeu. 4

ven. 5

sam. 6

Midi

0.5

1.0

1.3

100

1.00

100

1.00

100

1.00

100

1.00

100

1.00

100

1.00

Soir

0.5

1.0

1.3

100

1.00

100

1.00

100

1.00

100

1.00

100

1.00

100

1.00

PROPOSÉS

26

NON COUVERTS

5

Heures par équipier (après application)

Sophie

40h / 42h

Ahmed

40h / 42h

Elena

33.5h / 35h

Lucas

36.5h / 35h

Mehdi

34.5h / 35h

Clara

25.5h / 24h

Couverture par service et poste

dim. 31 — Q Midi Cuisine

1.55 / 1.00 (155%)

dim. 31 — I Midi Salle

1.40 / 1.00 (140%)

dim. 31 — Q Soir Cuisine

1.55 / 1.00 (155%)

dim. 31 — I Soir Salle

1.40 / 1.00 (140%)

mar. 2 — Q Midi Cuisine

0.95 / 1.00 (95%)

mar. 2 — I Midi Salle

0.80 / 1.00 (80%)

mar. 2 — Q Soir Cuisine

0.95 / 1.00 (95%)

Services non pourvus (en dessous du seuil mini)

mar. 2 — I Soir Salle : 40/100

mer. 3 — Q Midi Cuisine : 40/100

mer. 3 — I Midi Salle : 0/100

mer. 3 — Q Soir Cuisine : 40/100

mer. 3 — I Soir Salle : 0/100

✓ Appliquer (26 shifts)

Annuler

Modale Smart Planner — tableau (jour × service) de densité prévue avec presets Calme 0,5 / Normal 1,0 / Chargé 1,3, couverture par service et par poste, liste des services non couverts.

Bistrot du Vieux Port
Bonjour Sophie, vous êtes manager.

🔗 Ouvrir le planning

📅 HEURES PLANIFIÉES
261h
sur 248h cibles

🕒 SOUS-COUVERTURE
4

MEMBRES ACTIFS
8

📅 Mes prochains services

Planning de service
➔

- | | |
|-------------|----------------|
| mer. 3 juin | 🕒 Midi — Salle |
| mer. 3 juin | 🕒 Soir — Salle |
| jeu. 4 juin | 🕒 Midi — Salle |
| jeu. 4 juin | 🕒 Soir — Salle |
| ven. 5 juin | 🕒 Midi — Salle |

Heures cette semaine

JM **Julien Martin**
Administration **Admin**
22.5h /
0h
PLANIFIÉES

AK **Ahmed Kacimi**
Cuisine
50h /
42h
PLANIFIÉES

SB **Sophie Bernard**
Salle
50h /
42h
PLANIFIÉES

CP **Clara Petit**



Vue mobile responsive — application utilisable sur smartphone.

Spécifications techniques

I. Technologies

Developer experience & outils

- **Git / GitHub** (branches main + dev)
- **Node 22 + npm** — exécution et dépendances
- **Vite** — serveur de dev et build front
- **ESLint + Prettier** — qualité et formatage du code
- **Docker** — image de déploiement back (Fly.io)

Front-end

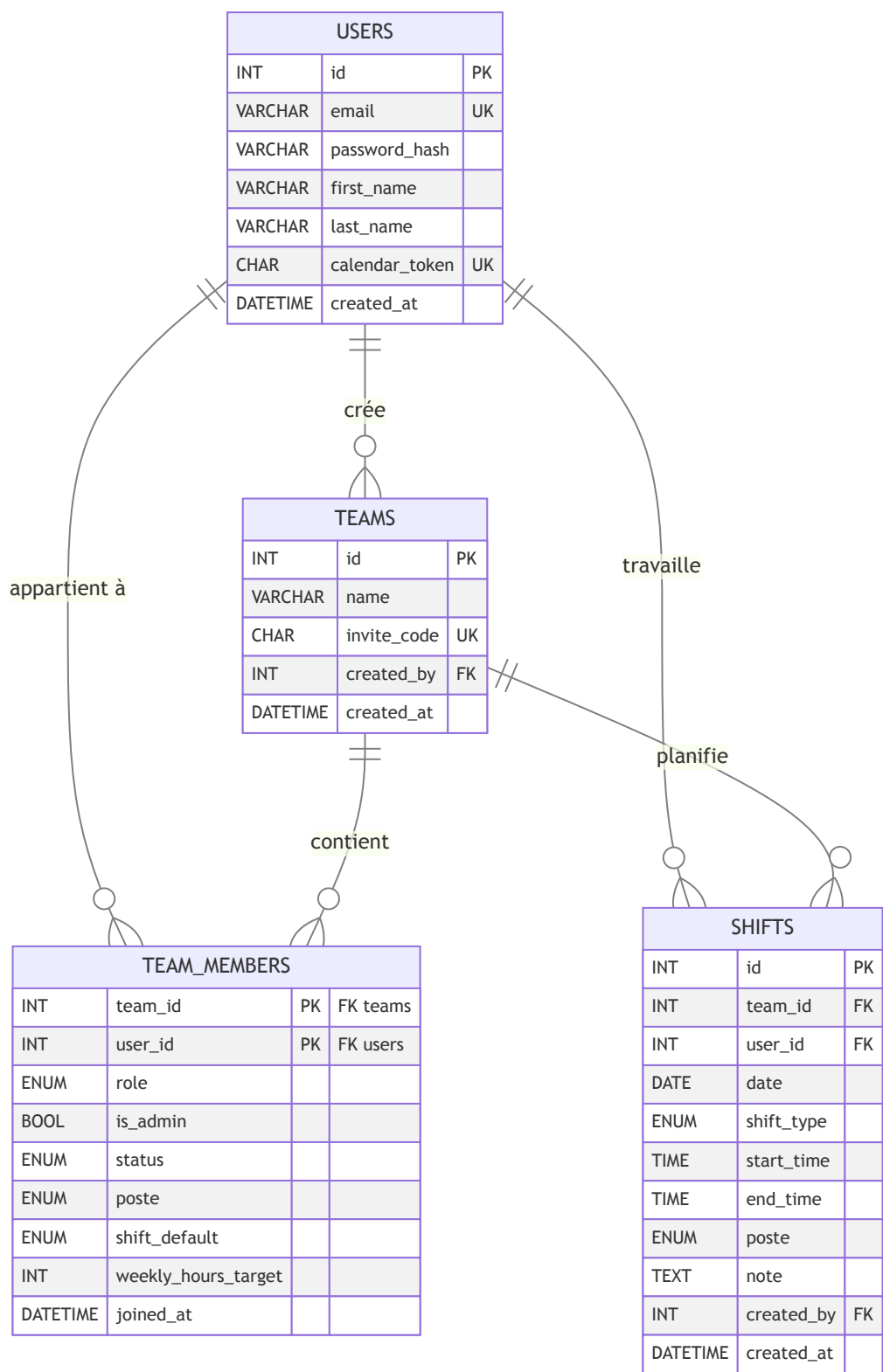
- **React 18** via Vite — composants fonctionnels et hooks
- **React Router 6** — navigation, routes protégées
- **i18next** — internationalisation FR/EN
- **Chart.js + react-chartjs-2** — statistiques
- **CSS pur (custom properties)** — design system responsive
- **Context API** — état global (auth, équipe, thème, toasts)

Back-end

- **Express 4** — routes, middlewares, contrôleurs (architecture MVC)
- **mysql2** — accès données, requêtes préparées (protection injection SQL)
- **MariaDB 10.11** — base relationnelle
- **Migrations & seed** — versionnage du schéma, données de démo
- **bcrypt, jsonwebtoken, express-rate-limit, cors** — sécurité
- **ical-generator** — flux calendrier RFC 5545

II. Création de la base de données

A. Modèle conceptuel / logique (MCD-MLD)



B. Dictionnaire des données

Table **users**

Donnée	Désignation	Type	Contrainte
id	Identifiant utilisateur	INT	PK, AUTO_INCREMENT
email	Adresse e-mail	VARCHAR(255)	NOT NULL, UNIQUE
password_hash	Mot de passe haché (bcrypt)	VARCHAR(255)	NOT NULL
first_name	Prénom	VARCHAR(80)	NOT NULL
last_name	Nom	VARCHAR(80)	NOT NULL
calendar_token	Token iCal opaque	CHAR(48)	NOT NULL, UNIQUE
created_at	Date de création	DATETIME	DEFAULT CURRENT_TIMESTAMP

Table **teams**

Donnée	Désignation	Type	Contrainte
id	Identifiant équipe	INT	PK, AUTO_INCREMENT
name	Nom de l'équipe	VARCHAR(120)	NOT NULL
invite_code	Code d'invitation (CREW-XXXX-XXXX)	CHAR(14)	NOT NULL, UNIQUE
created_by	Créateur	INT	FK → users(id), NOT NULL
created_at	Date de création	DATETIME	DEFAULT CURRENT_TIMESTAMP

Table **team_members** (association N-N enrichie)

Donnée	Désignation	Type	Contrainte
team_id	Équipe	INT	PK, FK → teams(id) ON DELETE CASCADE
user_id	Utilisateur	INT	PK, FK → users(id) ON DELETE CASCADE
role	Rôle	ENUM(manager, equipier)	DEFAULT 'equipier'
is_admin	Administrateur	BOOLEAN	DEFAULT FALSE
status	Statut d'adhésion	ENUM(active, pending)	DEFAULT 'pending'
poste	Poste habituel	ENUM(cuisine... administration)	NULL
shift_default	Créneau habituel	ENUM(matin...nuit)	NULL
weekly_hours_target	Heures cibles/semaine	INT	NULL (0/NULL = exclu solver)
joined_at	Date d'adhésion	DATETIME	DEFAULT CURRENT_TIMESTAMP

Table `shifts`

Donnée	Désignation	Type	Contrainte
id	Identifiant du shift	INT	PK, AUTO_INCREMENT
team_id	Équipe	INT	FK → teams(id) ON DELETE CASCADE
user_id	Équipier planifié	INT	FK → users(id) ON DELETE CASCADE
date	Jour du service	DATE	NOT NULL
shift_type	Type de créneau	ENUM(matin...nuit)	NOT NULL
start_time	Heure de début	TIME	NULL (dérivé du type sinon)
end_time	Heure de fin	TIME	NULL
poste	Poste tenu (renfort possible)	ENUM(cuisine...administration)	NOT NULL
note	Note libre	TEXT	NULL
created_by	Manager créateur	INT	FK → users(id) ON DELETE SET NULL
created_at	Création	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	Mise à jour	DATETIME	ON UPDATE CURRENT_TIMESTAMP

C. Règles d'intégrité & index

Règle	Mécanisme
Email unique	UNIQUE sur <code>users.email</code>
Token iCal unique	UNIQUE sur <code>users.calendar_token</code>
Code d'invitation unique	UNIQUE sur <code>teams.invite_code</code>
Pas de double-planning	UNIQUE (<code>user_id</code> , <code>date</code> , <code>shift_type</code>) sur <code>shifts</code>
Suppression équipe ⇒ cascade des shifts	FK <code>team_id</code> ON DELETE CASCADE
Historisation du créateur d'un shift	FK <code>created_by</code> ON DELETE SET NULL

Index principaux : `users(email)`, `users(calendar_token)`, `teams(invite_code)`, `team_members(user_id)`, `team_members(team_id, poste)` (solver), `shifts(user_id, date)`, `shifts(team_id, date)`.

Migrations appliquées séquentiellement par `npm run migrate` : 001_users → 002_teams → 003_team_members → 004_member_planning_fields → 005_shifts, puis 006-012 (évolutions, voir dernière section).

III. API REST — routes

Toutes les routes sont préfixées par `/api`. Sauf `/auth/*` et `/calendar/*`, toutes exigent un header `Authorization: Bearer <jwt>`.

Authentification & profil

Méthode	Chemin	Description	Accès
POST	/auth/register	Création de compte	Public
POST	/auth/login	Connexion → JWT	Public
GET	/users/me	Profil connecté	Auth
PATCH	/users/me	Modifier prénom/nom	Auth
POST	/users/me/password	Changer son mot de passe	Auth

Équipes & membres

Méthode	Chemin	Description	Accès
GET	/teams	Mes équipes	Auth
POST	/teams	Créer une équipe	Auth
GET	/teams/:id	Détail + membres	Membre
POST	/teams/join	Rejoindre via code	Auth
POST	/teams/:id/approve/:userId	Valider un membre	Admin
PATCH	/teams/:id/members/:userId	Modifier rôle/poste/heures	Admin
DELETE	/teams/:id/members/:userId	Retirer un membre	Admin
POST	/teams/:id/regenerate-code	Régénérer le code	Admin

Shifts & smart planner

Méthode	Chemin	Description	Accès
GET	/shifts?team_id&from&to	Lister les shifts	Membre
GET	/shifts/upcoming	Mes 10 prochains shifts	Auth
POST / PUT / DELETE	/shifts(/:id)	CRUD d'un shift	Manager
GET	/shifts/summary	Heures/membre + slots non couverts + coût	Manager
POST	/shifts/generate-plan	Proposer un planning (preview)	Manager
POST	/shifts/apply-plan	Créer en base les shifts proposés	Manager
POST	/shifts/clone-week	Dupliquer une semaine	Manager
DELETE	/shifts/clear-week	Vider une semaine	Manager

Statistiques & calendrier

Méthode	Chemin	Description	Accès
GET	/stats/dashboard	Tableau de bord	Auth
GET	/stats/charts?team_id	Stats équipe (poste, membre, timeline)	Manager
GET	/calendar/:token	Flux iCal de tous les shifts	Public (token)
GET	/calendar/:token/team/:teamId.ics	Flux iCal d'une équipe	Public (token)

Extraits de code commentés

Ces extraits illustrent les compétences clés du titre. Chacun est commenté pour expliquer le choix technique et la mesure de sécurité associée.

1. Back-end — authentification JWT (middleware)

Chaque route protégée passe par ce middleware. Il refuse l'accès si le header est absent/mal formé, vérifie la signature du token et n'expose au reste de l'application que l'identifiant utilisateur. **Compétence II.C — sécurité back.**

```
// back/src/middleware/auth.js
import { verifyToken } from '../services/jwt.service.js';
import { unauthorized } from '../utils/httpError.js';

export function authRequired(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    return next(unauthorized()); // 401 : pas de token
  }
  try {
    const payload = verifyToken(header.slice(7)); // vérifie la signature HS256
    req.user = { id: payload.sub }; // on n'expose que l'id
    next();
  } catch {
    next(unauthorized('Token invalide ou expiré'));
  }
}
```

2. Back-end — hachage du mot de passe (bcrypt)

Les mots de passe ne sont jamais stockés en clair : bcrypt (cost 10) produit un hash salé. La comparaison est faite par bcrypt lui-même (résistant au timing). **Compétence II.C.**

```
// back/src/services/password.service.js
import bcrypt from 'bcrypt';
const ROUNDS = 10;

export function hashPassword(plain) {
  return bcrypt.hash(plain, ROUNDS); // salt + hash automatiques
}
export function comparePassword(plain, hash) {
  return bcrypt.compare(plain, hash);
}
```

3. Back-end — contrôle d'accès par rôle (RBAC)

Au-delà de l'authentification, l'autorisation est vérifiée par middleware sur chaque endpoint sensible : appartenance à l'équipe, rôle manager, statut administrateur. **Compétence II.C.**

```
// back/src/middleware/teamAccess.js
export async function requireTeamMember(req, res, next) {
  const teamId = Number(req.params.teamId || req.body.team_id);
  if (!teamId) return next(forbidden('team_id manquant'));
  const member = await teamMemberModel.findByTeamAndUser(teamId, req.user.id);
  if (!member || member.status !== 'active')
    return next(forbidden("Vous n'êtes pas membre de cette équipe"));
  req.teamMember = member;
  next();
}

export function requireManager(req, res, next) {
  if (!req.teamMember || req.teamMember.role !== 'manager')
    return next(forbidden('Réservé aux managers'));
  next();
}
```

4. Back-end — accès aux données en requêtes préparées

La couche Model n'écrit jamais de SQL par concaténation : toutes les valeurs passent par des paramètres liés [?](#) (driver mysql2), ce qui neutralise l'injection SQL. **Compétence II.B — accès aux données + sécurité.**

```
// back/src/models/user.model.js
export const userModel = {
  async findByEmail(email) {
    const [rows] = await pool.query(
      'SELECT * FROM users WHERE email = ?', [email]); // paramètre lié
    return rows[0] || null;
  },
  async create({ email, password_hash, first_name, last_name, calendar_token }) {
    const [result] = await pool.query(
      `INSERT INTO users (email, password_hash, first_name, last_name, calendar_token)
      VALUES (?, ?, ?, ?, ?)`,
      [email, password_hash, first_name, last_name, calendar_token]);
    return result.insertId;
  },
};
```

5. Back-end — validation systématique des entrées

Toute donnée venant du client est validée côté serveur avant traitement (format email, longueur du mot de passe, champs requis). La validation lève une erreur 400 structurée. **Compétence II.C.**

```
// back/src/validators/auth.validator.js
const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

export function validateRegister(body) {
  const errors = {};
  const { email, password, first_name, last_name } = body || {};
  if (!email || !EMAIL_RE.test(email) || email.length > LIMITS.EMAIL_MAX)
    errors.email = 'Email invalide';
  if (!password || password.length < LIMITS.PASSWORD_MIN)
    errors.password = `Mot de passe d'au moins ${LIMITS.PASSWORD_MIN} caractères`;
  if (!first_name) errors.first_name = 'Prénom requis';
  if (!last_name) errors.last_name = 'Nom requis';
  if (Object.keys(errors).length) throw badRequest('Champs invalides', errors);
  return { email, password, first_name, last_name };
}
```

6. Logique métier — cœur du solver (scoring)

Le composant métier central : pour chaque créneau à couvrir, le solver filtre les candidats éligibles (compétence, disponibilité, **plafonds légaux HCR jamais relâchés**), puis les classe par un score multicritères. **Compétence II.C — composant métier.**

```
// back/src/services/plannerSolver.js (extrait de findBest)
.filter((m) => {
  if (!m.weekly_hours_target) return false; // exclu du solver
  if (!canFill(m, slot.poste)) return false; // compétence/poste
  if (wouldBeWeek > HCR_WEEKLY_MAX) return false; // 48 h/sem – mur légal
  if (dayHours + shiftDur > hcrDailyCap(m.poste)) return false; // plafond/jour
  if (countConsecutive([...userDays, slot.date]) > MAX_CONSECUTIVE_DAYS)
    return false; // max 6 jours consécutifs
  return true;
})
.map((m) => {
  const deficit = m.weekly_hours_target - hours[m.user_id].planned;
  let score = deficit * 10; // priorité : qui manque d'heures
  if (m.shift_default === slot.service) score += 5; // créneau habituel
  if (m.poste === slot.poste) score += 3; // poste habituel
  if (m.level === 'chef') score += 2; // séniorité
  score -= (shiftCost(m, slot.service, cfg) / 100) * COST_PENALTY_WEIGHT; // coût
  return { member: m, score };
})
.sort((a, b) => b.score - a.score)[0]?.member ?? null;
```


Sécurité — analyse OWASP

La sécurité est traitée à chaque niveau de l'application. Le tableau ci-dessous met en regard les principaux risques (référentiel OWASP Top 10) et les mesures concrètes implémentées dans Crew.

Risque OWASP	Mesure dans Crew	Emplacement
A01 — Broken Access Control	RBAC par middleware sur chaque endpoint (membre / manager / admin) ; vérification d'appartenance à l'équipe avant toute action.	<code>middleware/teamAccess.js</code>
A02 — Cryptographic Failures	Mots de passe hachés avec bcrypt (cost 10, salt) ; jamais stockés ni renvoyés en clair ; HTTPS forcé en production.	<code>services/password.service.js</code>
A03 — Injection	Requêtes SQL exclusivement préparées (paramètres liés mysql2) ; aucune concaténation de chaîne dans les requêtes.	<code>models/*.model.js</code>
A04 — Insecure Design	Validation systématique côté serveur ; principe du moindre privilège ; le token client ne contient que l'id (aucun secret).	<code>validators/*</code>
A05 — Security Misconfiguration	CORS restrictif (liste blanche d'origines via <code>FRONT_ORIGIN</code>) ; <code>trust proxy</code> pour la détection d'IP derrière Fly ; secrets en variables d'environnement.	<code>app.js</code> , <code>config/env.js</code>
A07 — Identification & Auth. Failures	JWT signé HS256 avec expiration ; rate-limiting des tentatives de connexion et d'inscription ; messages d'erreur sans fuite d'information.	<code>middleware/auth.js</code> , <code>rateLimit.js</code>
A09 — Logging & Monitoring	Journalisation des requêtes (requestLogger) et gestion centralisée des erreurs (errorHandler) pour la traçabilité.	<code>middleware/requestLogger.js</code>

Sécurité côté front-end

- **Routes protégées** — un composant `ProtectedRoute` redirige vers la connexion si aucun token valide n'est présent.
- **Aucun secret exposé** — le client ne manipule que le JWT ; aucune clé serveur, aucun identifiant de base n'est embarqué dans le bundle.
- **Validation des saisies** — les formulaires (inscription, connexion, changement de mot de passe) contrôlent les entrées avant envoi et affichent des retours d'erreur clairs.
- **Échappement** — React échappe par défaut le contenu rendu, neutralisant les injections XSS basiques.

Rate-limiting (extrait)

```
// back/src/middleware/rateLimit.js
export const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: isProd ? 20 : 1000,          // 20 essais / 15 min en prod : rempart anti brute-force
  message: { error: 'Trop de tentatives, réessayez dans 15 minutes' },
});
```

Plan de tests & jeu d'essai

Couverture manuelle exécutée avant chaque mise en production et avant la soutenance, complétée par un jeu de données de démo (seed) couvrant chaque rôle.

Comptes de test (seed)

Email	Rôle	Mot de passe
julien.patron@bistrot.fr	Patron (admin)	motdepasse123
sophie.manager@bistrot.fr	Manager salle	motdepasse123
ahmed.chef@bistrot.fr	Chef cuisine	motdepasse123
elena.serveuse@bistrot.fr	Serveuse	motdepasse123
samir.plonge@bistrot.fr	Plongeur	motdepasse123

1. Authentification

#	Scénario	Résultat attendu	Statut
1.1	Inscription valide (mdp ≥ 8)	Redirection dashboard, JWT stocké	OK
1.2	Inscription email déjà utilisé	Erreur 409, message clair	OK
1.3	Mot de passe trop court	Erreur 400 (validateur)	OK
1.4	Connexion valide	Redirection dashboard	OK
1.5	Mauvais mot de passe	401, pas de fuite d'info	OK
1.6	Rate limit login (excès de tentatives)	Réponse 429	OK
1.7	Token expiré sur route protégée	401, redirect login	OK
1.8	Changement de mot de passe	Succès, ré-auth requis	OK

2. Équipe & permissions

#	Scénario	Résultat attendu	Statut
2.1	Création d'équipe	Équipe créée, créateur = admin	OK
2.2	Adhésion par code valide	Membre en statut <code>pending</code>	OK
2.3	Adhésion par code invalide	Erreur 404	OK
2.4	Équipier tente d'éditer un shift	403 (RBAC)	OK
2.5	Non-membre accède au détail équipe	403	OK

3. Smart Planner

#	Scénario	Résultat attendu	Statut
3.1	Génération sur le seed	Preview cohérente, slots non couverts signalés	OK
3.2	Respect du plafond 48 h/semaine	Aucun équipier au-dessus de 48 h	OK
3.3	Max 6 jours consécutifs	Aucune affectation au 7 ^e jour consécutif	OK
3.4	Application du plan	Shifts créés en base, unicité respectée	OK
3.5	Clone-week / clear-week	Semaine dupliquée / vidée	OK

4. Calendrier natif

#	Scénario	Résultat attendu	Statut
4.1	Abonnement au flux iCal perso	Shifts visibles dans le calendrier	OK
4.2	Alarme VALARM	Notification 2 h avant le service	OK
4.3	Token erroné	Flux vide / 404, pas de fuite	OK

Suite prévue : ajouter une couche de tests automatisés (Vitest + supertest) sur le solver (plafonds HCR, scoring) et l'authentification, pour compléter cette couverture manuelle.

Veille technique & exemple de raisonnement

Pour les décisions techniques importantes, j'applique une démarche de veille : formuler la question, comparer 2-3 sources fiables (dont anglophones), évaluer selon des critères, tester, puis décider — en gardant une alternative de secours.

Exemple — algorithme du solver : heuristique greedy vs solveur exact (MIP)

Contexte

Le cœur de Crew est l'affectation d'équipiers à des créneaux sous contraintes (couverture cible, heures contractuelles, plafonds légaux HCR). C'est un problème classique de *staff scheduling / rostering*. Deux familles d'approches existent :

- **Solveur exact (MIP)** — modéliser le problème en programme linéaire en nombres entiers et le confier à un solveur (couverture en contrainte, coût minimisé en objectif).
- **Heuristique greedy** — construire la solution créneau par créneau en choisissant à chaque étape le meilleur candidat selon un score.

Recherche comparative (sources anglophones)

Je me suis appuyé sur deux références de recherche opérationnelle en anglais :

Ernst, A. T., Jiang, H., Krishnamoorthy, M. & Sier, D. (2004). « *Staff scheduling and rostering: A review of applications, methods and models* », *European Journal of Operational Research* 153(1):3-27.

Bard, J. F., Binici, C. & deSilva, A. H. (2003). « *Staff scheduling at the United States Postal Service* », *Computers & Operations Research* 30(5):745-771.

Cette littérature établit que (1) la planification du personnel se modélise classiquement par minimisation de la masse salariale sous contrainte de couverture, et que (2) le coût horaire est l'objectif dominant dans plus de 95 % des modèles appliqués recensés. Le MIP donne l'optimum global, mais au prix d'un temps de calcul non borné et d'une « boîte noire » difficile à expliquer à un manager.

Critère	Solveur exact (MIP)	Heuristique greedy
Qualité de la solution	Optimum global	Quasi-optimale (bonne en pratique)
Temps de réponse	Variable, peut exploser	Temps réel, borné
Explicabilité	Faible (boîte noire)	Forte (score lisible)
Dépendance externe	Solveur tiers à intégrer	Aucune (JS pur)
Gestion des contraintes dures	Native	Par filtrage explicite

Décision & justification

J'ai retenu une **heuristique greedy avec scoring multicritères**, pour trois raisons :

1. **Temps réel** — la preview doit s'afficher instantanément quand le manager clique.

2. **Explicabilité** — le score (déficit d'heures, poste, séniorité, coût) est lisible et défendable devant un manager, là où un MIP est opaque.
3. **Contraintes dures garanties** — les plafonds légaux HCR (48 h/semaine, 6 jours max, repos hebdomadaire) sont appliqués par filtrage strict *avant* le scoring : aucune suggestion ne peut les enfreindre, exactement comme une contrainte de MIP.

La même philosophie que Bard et al. (couverture imposée, coût pénalisé) est donc conservée, sous une forme heuristique adaptée à un usage interactif. La limite assumée — une solution quasi-optimale plutôt qu'optimale — est acceptable car le manager ajuste toujours la proposition avant publication.

Évolutions — solver enrichi (migrations 006-012)

Au-delà du périmètre MVP, le projet a intégré six dimensions métier supplémentaires, documentées et adossées à la littérature peer-reviewed (annexe scientifique, 12 références).

Profils normalisés (006)

Chaque équipier porte un **niveau** (junior / confirmé / chef) auquel l'UI applique 5 profils nommés (Apprenti 0,15 → Référént 0,60). Tous les poids sont < 1 ; chaque poste vise une couverture de 1,00 (= 100 %).

Paramètres d'établissement (007)

La table `teams` reçoit des coefficients par niveau, une capacité de référence (`max_couverts`) et des idéaux de couverture par service et par poste, éditables par le manager. Les anciennes constantes en dur sont supprimées.

Override personnel & coverage agrégée (008)

`team_members.coef_override` permet de surcharger le poids d'un équipier au cas par cas. L'API `summary` expose la couverture par poste.

Jours d'ouverture & densité prévue (010)

`teams.closed_days_mask` (bitmask 7 bits) remplace l'ancienne constante. La modale Smart Planner expose un tableau (date × service) où le manager déclare la densité prévue (Calme 0,5 / Normal 1,0 / Chargé 1,3). Le solver scale l'idéal par cette densité.

Polyvalence — multi-skill (011)

`team_members.skills_mask` (bitmask des postes maîtrisés). Le solver lit ce masque en priorité ; le scoring conserve un bonus `+3` pour le poste primaire, si bien qu'un équipier polyvalent **n'est mobilisé hors de sa spécialité que si aucun spécialiste primaire n'est éligible**. Application du *théorème de la chaîne courte* (Jordan & Graves 1995, *Management Science*) : 2-3 postes/équipier captent ~95 % des gains d'une flexibilité totale.

Optimisation économique cost-aware (012)

Trois colonnes `*_rate` sur `teams` + `rate_override` , alignées sur les minima de la Convention HCR 2026 (Junior 12 €, Confirmé 14 €, Chef 19 €). Le solver intègre une pénalité de coût dans le score, calibrée pour rester sous-dominante face au déficit hebdomadaire — le coût ne discrimine qu'entre candidats à besoins équivalents. La masse salariale prévisionnelle (`laborCostTotal`) s'affiche dans l'en-tête de la page Planning.

Alertes science-based intégrées

- **fatigueAlerts** — surcharge soutenue (KC & Terwiesch 2009, *Management Science*).
- **hcrViolations** — non-conformités Convention HCR / Code du travail (Matre et al. 2021).
- **serviceHealth** — score composite 0-100 par (jour, service), pastille Saine / Tendue / Risque.

Récapitulatif des migrations

#	Sujet	Tables impactées
006	Niveaux Junior / Confirmé / Chef	team_members
007	Paramètres d'équipe (coef, idéal, capacité)	teams
008	Coef override personnel	team_members
009	Normalisation [0;1] des poids et idéaux	teams, team_members
010	Jours d'ouverture (closed_days_mask)	teams
011	Polyvalence (skills_mask)	team_members
012	Taux horaires + override	teams, team_members

Justification scientifique complète et bibliographie : annexe [JUSTIFICATION_SCIENTIFIQUE.md](#) (12 références peer-reviewed et institutionnelles).

Conclusion

La réalisation de Crew m'a permis de transformer un besoin métier concret — la planification d'équipe sous contraintes — en une application web complète, fonctionnelle et déployée en production.

Ce projet m'a confronté à l'ensemble des compétences du titre : conception d'une base de données relationnelle et de ses contraintes d'intégrité, développement d'une API REST structurée en MVC, réalisation d'une interface React responsive et internationalisée, et — au cœur du projet — la conception d'un composant métier non trivial, le solveur d'auto-planning, qui applique des règles légales strictes tout en optimisant l'équité et le coût.

J'ai particulièrement soigné la **sécurité à chaque niveau** : authentification JWT, hachage bcrypt, contrôle d'accès par rôle sur chaque endpoint, requêtes préparées, validation systématique des entrées, CORS restrictif et rate-limiting. Ces choix sont documentés et mis en regard du référentiel OWASP.

À chaque décision technique importante, j'ai adopté une démarche de veille — comparer des sources fiables, dont anglophones, avant de trancher — comme l'illustre le choix entre solveur exact et heuristique greedy.

Le projet reste perfectible : j'identifie comme suites prioritaires l'ajout d'une couche de tests automatisés, des notifications push, et une suggestion automatique de la densité de service à partir de l'historique. Ces limites sont assumées et tracées.

Au terme de ce travail, je retiens surtout une montée en compétence sur la conception d'un système complet — du modèle de données à la mise en production — et la capacité à mener un projet technique exigeant de façon autonome et rigoureuse, dans un cadre proche d'un contexte professionnel.